Calibración de un centellador plástico por Compton edge

El análisis implementado, paso a paso, contra el código que corre.
Método NCC + recocido simulado (Kong et al., IEEE TNS 2025, arXiv:2507.16282), adaptado a un centellador plástico leído con SiPM.
Generado automáticamente el 02-09-2026 por `codigo/documentacion.py`.

Este documento describe **el estado actual** del análisis en orden de ejecución: qué le pasa a los datos desde el XML del osciloscopio hasta el número final, qué archivo y qué función hace cada cosa, qué decisiones de diseño hay detrás y qué quedaría por mejorar en cada paso. No es el cuaderno de laboratorio (eso es `CLAUDE.md`, que crece por el final y mezcla el hallazgo con su corrección posterior): acá está lo que hoy hace el código.

Los números no están tipeados. Se calculan al generar el PDF: las constantes se leen de los módulos, las figuras salen de los datos reales y el resultado sale de correr el ajuste. Si mañana cambia `spectrum.GATE_POST`, el documento de la próxima corrida lo dice.

Sobre qué datos se generó este documento

	archivo	qué es
Muestra	A Co60 5GaS 50k 02mV trigg 1550 att.xml	corrida con la fuente
Fondo	A BG 5GaS 10k 02mV trigg 1550 att.xml	corrida sin fuente, se resta
Simulación	ComptonSpectrum_Co60_3M.root	plantilla de Geant4
Fuente	Co-60	líneas gamma: 1173.2 y 1332.5 keV

El resultado, para tenerlo a mano

cantidad	valor	de dónde sale
D (luz)	1.2618 ± 0.0137 mV·ns/keVee	`modelo_ext` con plantilla `Lvis_keVee`
D (edep)	1.2091 ± 0.0144 mV·ns/keV	lo mismo con `Edep_keV`
resolución	σ_E = 2.764·√E keV → 21.0% FWHM a 963 keV	el parámetro α del mismo ajuste
mezcla w	0.797 ± 0.072 (la simulación trae 0.751)	fracción de scattering simple, ajustada
bondad	χ^2/ndf = 0.998 con 324 grados de libertad	χ^2 de Pearson iterado

El eje horizontal de todo el proyecto es el **área del pulso en mV·ns**, no un canal de ADC. `D` es la pendiente que lo convierte a energía, y el mapa pasa por el origen: área cero ↔ energía cero.

0. El panorama en una página

El problema: un centellador plástico es de bajo Z, así que domina el efecto Compton y **no hay fotopico**. La calibración clásica —poner el pico de energía conocida en su canal— no aplica. Lo que sí hay es el **borde Compton**: la energía máxima que un gamma le puede dar a un electrón, que ocurre en retrodispersión y vale

$$E_{max} = 2\cdot\alpha\cdot E_\gamma / (1 + 2\cdot\alpha), \qquad \alpha = E_\gamma / m_e c^2 \quad (m_e c^2 = 510.999\ \text{keV})$$

Esa energía es conocida sin ajustar nada. Si uno logra decir en qué área de pulso cae ese borde, tiene un punto de calibración. Todo el resto del análisis es responder esa pregunta con cuidado, porque el borde no es un pico: es un escalón borroneado por la resolución del detector y contaminado por física que la fórmula no contiene.

La cadena completa

#	paso	archivo	qué produce
1	XML → área del pulso	`datos_io/spectrum.py`	una área en mV·ns por evento
2	áreas → espectro neto	`spectrum.espectro_neto`	histograma con fondo restado y barras de error
3	física teórica	`etapas/physics.py`	E_max y la forma dσ/dE_e
4	forma → espacio de canal	`etapas/model.py`	el modelo que se compara con los datos
5	elegir la ventana	`etapas/ncc.py`	el intervalo de ajuste y la semilla, sin criterio humano
6	ajustar	`etapas/fit_single.py`, `etapas/fit_double.py`	D por la ruta analítica
7	optimizador global	`etapas/sa.py`	verificar que el óptimo es global
8	plantilla de Geant4	`ajuste/sim_template.py`	reemplaza la teoría analítica por la simulación
9	modelo extendido	`ajuste/modelo_ext.py`	**el resultado vigente**: sistemáticos como parámetros
10	posterior	`ajuste/mcmc.py`	barras de error y correlaciones reales
11	resultado	`calibracion.py`, pestaña ★	D, su presupuesto de error y el registro por fuente

Los pasos 3 a 7 son la réplica del paper. Los pasos 8 a 11 son lo que hubo que agregarle para que el número fuera creíble en **este** detector, y son también donde está casi todo el error sistemático.

Dónde vive cada cosa

```
Calibración CE/
├─ analisis_paso_a_paso.pdf   este documento
├─ CLAUDE.md                  el cuaderno de laboratorio
├─ codigo/
│   ├─ rutas.py               dónde está cada cosa
│   ├─ gui.py  calibracion.py  documentacion.py
│   ├─ etapas/    physics · model · fit_single · fit_double · ncc · sa
│   ├─ datos_io/  spectrum
│   ├─ ajuste/    sim_template · modelo_ext · mcmc
│   └─ scripts/   calibrar_v2 · calibrar_ncc · calibrar
├─ datos/       los .xml del DRSOSC, sus cachés .npz y los .root
├─ figuras/     lo que los módulos dibujan al correr solos
├─ docs/        el paper de referencia y notas históricas
├─ verificaciones/  scripts sueltos de comprobación
└─ .cache/      preferencias de la GUI, cadenas MCMC, registro
```

Cada módulo **corre solo** y deja su figura en `figuras/`: `python codigo/etapas/ncc.py` muestra la Etapa 5 sobre datos sintéticos con verdad conocida, y así con los demás. Es la forma prevista de tocar una etapa sin arrastrar el resto. Todas las rutas se resuelven desde `__file__` vía `codigo/rutas.py`, así que no importa desde qué carpeta se ejecute.

1. Del XML crudo al área del pulso

Archivo: `datos_io/spectrum.py`. **Funciones:** `detectar_canal`, `_iter_eventos`, `_area_pulso_gate`, `tiempo_vivo`, `leer_areas_y_tiempo`.

El DRSOSC guarda un XML con un tag por línea y la estructura `DRSOSC > Event > Board_XXXX > CHN{n} > Data`, donde cada `Data` es un par «t,v» con t en ns y v en mV. El archivo de la muestra pesa 1.76 GB y tiene del orden de $5{\cdot}10^7$ tags `Data`.

1.1 Leer el archivo sin morir en el intento

`ElementTree` construye un objeto por tag: decenas de GB de RAM y varios minutos. Como el formato del DRSOSC es rígido, `_iter_eventos` lo recorre **como texto**, línea por línea, y convierte cada pulso de una sola vez con `np.fromstring(",".join(pares), sep=",")`. La RAM queda plana y el archivo grande se lee en unos 30 s.

Dos cosas se detectan en vez de asumirse. `detectar_canal` mira el primer evento y devuelve qué `CHN{n}` y qué `Scaler{n-1}` trae: las corridas de junio salieron por `CHN2` y las de agosto por `CHN1`, y tener el canal cableado hacía que se recorrieran los 2.9 GB encontrando **cero** pulsos, en silencio. Además el resultado se cachea en `<archivo>.areas_gate.npz` con una huella que incluye tamaño, fecha, umbral y canal, así que la segunda corrida es instantánea.

> **Trampa que ya costó cara:** el caché guardaba también los resultados vacíos. Hoy `leer_areas_y_tiempo` levanta `ValueError` si quedan 0 pulsos útiles, en vez de dejar un `.npz` de ceros que reaparece cinco etapas más abajo como un `IndexError` incomprensible.

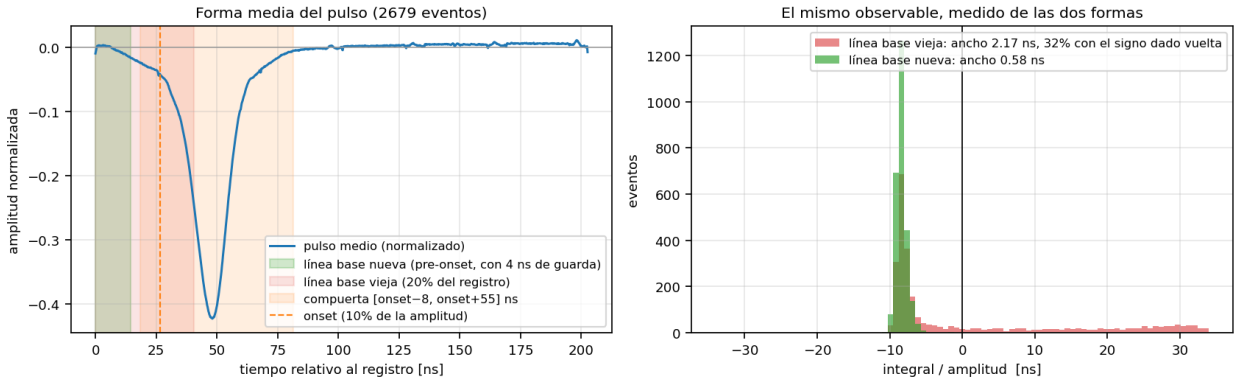
1.2 El área: línea base fuera del pulso y compuerta

Este es **el paso más importante de todo el proyecto**, y el que más tiempo estuvo mal. `_area_pulso_gate` hace, por evento:

- línea base provisoria = mediana de todo el registro (el pulso ocupa ~30% de las muestras, así que la mediana no lo ve);
- pico = máximo de |v|; **onset** = último cruce del 10% de la amplitud antes del pico;
- compuerta = `[onset − 8 ns, onset + 55 ns]`;
- línea base definitiva = **mediana de las muestras pre-onset**, dejando 4 ns de guarda antes de que abra la compuerta;
- área = integral trapezoidal dentro de la compuerta, con esa línea base restada.

Todo se mide contra `t − t[0]`, nunca contra t absoluto: el DRSOSC escribe el eje relativo a la celda de trigger y en ~2% de los eventos arranca corrido.

> **Por qué esto importa tanto:** la versión anterior promediaba la línea base sobre el **primer 20% del registro** (40 ns) e integraba los 200 ns completos. Pero el pulso arranca a los ~25 ns: la ventana de línea base se comía la subida del propio pulso. El sesgo se multiplica por $T/T_b \approx 5$ al integrar todo, y como el jitter del trigger mueve el pico entre 18 y 54 ns, **cada evento perdía una fracción distinta de su carga** — un ensanchamiento multiplicativo puro. Una fracción grande de los eventos salía incluso con el área de **signo invertido**, y un `abs()` los reciclaba como buenos (la figura de abajo lo mide sobre estos datos). Arreglarlo cambió D por un factor 3.4 y bajó la resolución de 42.7% a ~21% FWHM. El panel derecho de la figura es esa comparación, sobre los datos reales.



Izquierda: forma media del pulso con las dos definiciones de línea base y la compuerta. Derecha: el cociente integral/amplitud, que a resolución perfecta sería una constante. La distribución roja es lo que producía la línea base vieja.

1.3 Banderas de calidad

`_area_pulso_gate` devuelve una bandera por evento, un bit por problema: **2** = saturación del digitalizador (|v| ≥ 495 mV), **4** = el registro arranca dentro del pulso (queda truncado), **8** = no había pre-pulso suficiente y la línea base salió del complemento de la compuerta.

El bit 8 **no descarta**: el evento sirve, sólo tiene el área ~1.6% alta. La razón es sutil y vale la pena tenerla presente: descartar por posición del onset sería un corte **dependiente de la amplitud**, porque el trigger tiene walk (los pulsos grandes cruzan el umbral ~12 ns antes que los chicos). Un corte así deforma el espectro, que es exactamente lo que no se puede hacer en una calibración.

> **Para mejorar:** el pedestal del bit 8 (~6 mV·ns sobre la compuerta) se traduce casi 1:1 en D porque el mapa pasa por el origen. Hoy se marca y se acepta. Se podría corregir evento a evento en vez de tolerarlo, o cuantificar cuánto de D viene de ahí.

1.4 Tiempo vivo

El `<Scaler>` de cada evento es la tasa de trigger **real** de la placa. `tiempo_vivo` la integra y la compara con los eventos efectivamente guardados:

```
fracción viva = eventos guardados / ∫ scaler dt
T_vivo        = fracción viva × duración de pared
```

En estos datos la muestra estuvo viva **283 s de 699 s de pared (40%)** y el fondo **986 s de 986 s (100%)**: el DRSOSC vuelca ~35 kB de XML por evento y no da abasto a tasa alta. Normalizar por tiempo de pared, que es lo que se hacía antes, sobre-restaba el fondo por un factor 2.5.

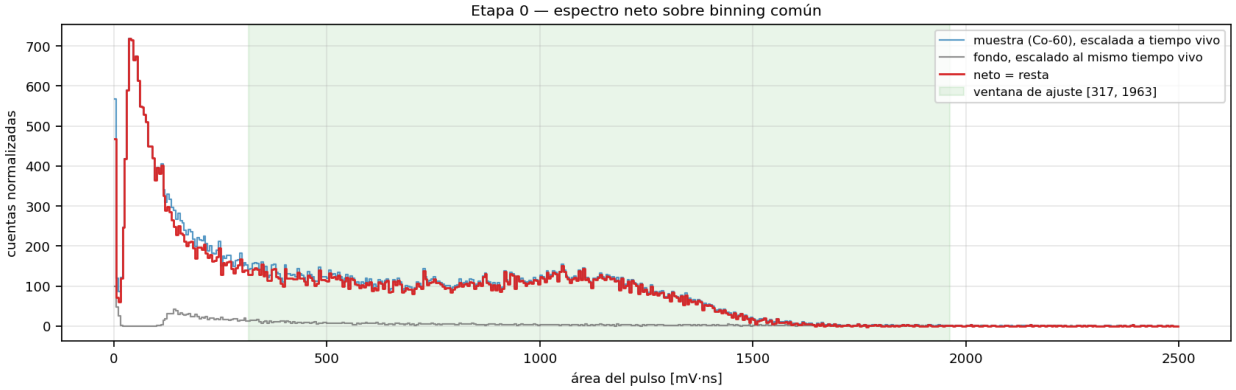
Las marcas de `<Time>` **no** sirven para esto: el DRSOSC las escribe en ráfagas y miles de eventos comparten el timestamp al milisegundo.

2. El espectro neto

Función: `spectrum.espectro_neto`. Toma las áreas de las dos corridas y devuelve el histograma listo para ajustar.

- **Binning común obligatorio.** Muestra y fondo se histograman con el mismo `range` fijo. Sin eso la resta es físicamente inválida: bins distintos serían mV·ns distintos.
- `RANGO_MAX` **fijo, nunca automático.** Unos pocos eventos de área enorme (pile-up, cósmicos) estiran el máximo y dejan al borde con pocos bins. Hoy el default de la interfaz es 2500 mV·ns con 500 canales, y la pestaña de Archivos sugiere 1.6·p99 mirando los datos.

- **Normalización por tiempo vivo**, con los pesos `k_m` = 1.000 y `k_f` = 0.287 con que entra cada corrida.
- **Barras de error de Poisson propagadas por la resta:** `error = √(N_m·k_m² + N_f·k_f²)`, con piso 1 para que un bin vacío no tenga peso infinito.
- **El descarte por calidad se aplica igual a muestra y fondo**, si no la resta deja de significar algo. Acá se descartaron 5424 eventos de la muestra y 1654 del fondo.
- `recortar_negativos=False` **para ajustar** (el neto real, con sus fluctuaciones negativas incluidas); `True` sólo para graficar.



El espectro neto sobre el que se ajusta todo. La banda verde es la ventana que eligió el NCC (paso 5).

> **Un detalle que sigue abierto:** hay un **exceso de cuentas a bajo canal** que la simulación no tiene, y que no es el umbral de trigger (es un exceso, no un déficit). Candidatos: dispersión en el material de alrededor y en el portafuente, que la simulación no modela, o apilamiento de dark counts del SiPM. El fondo ya está restado, así que no es eso. Desde la Etapa 9 se **modela** en vez de esquivarlo (paso 9), pero la forma que se le asume es hoy el sistemático más grande.

3. La física teórica: E_max y la lineshape

Archivo: `etapas/physics.py`. Es pura teoría: no toca datos y trabaja siempre en keV.

- `compton_edge_energy(E_gamma)` — la Ec. 1 del paper, **corregida**.
- `dsigma_dE(E_e, E_gamma, A)` — Klein-Nishina (Ec. 2) pasada a energía del electrón vía el jacobiano, o sea la Ec. 10. Vale 0 fuera de (0, E_max): arriba del borde no hay cinemática, y ese corte **es** el escalón que se busca.
- `FUENTES`, `NOTAS_FUENTE` y la clase `Fuente` — qué líneas gamma tiene lo que se midió, sus bordes, el ancla `E_ref` y si el modelo de dos bordes aplica.

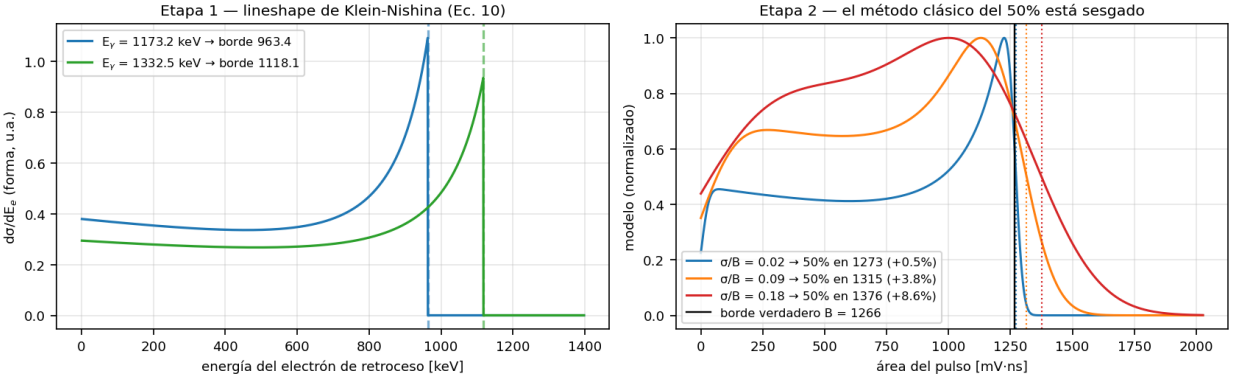
> **Errata del paper (1 de 3):** la Ec. 1 impresa, E_γ/(1+2α), es la energía del **fotón retrodispersado**, no la del electrón. La correcta lleva 2α en el numerador. Se verificó contra los propios datos del paper: con la fórmula corregida el Co-60 da 963 y 1118 keV, que es lo que muestra su Fig. 3.

En `A` se absorbe el prefactor π·r0²/(m_e c²) de la Ec. 10: para correlacionar y ajustar sólo importa la forma.

La fuente es un parámetro, no una constante

Todo lo que depende de qué se midió sale de un objeto `Fuente`, que se elige en la interfaz. De él salen las líneas, los bordes, el ancla `E_ref` del mapa por el origen y si hay uno o dos bordes que ajustar. Con la fuente cargada acá (Co-60: 1173.2 y 1332.5 keV -> bordes 963.4 y 1118.1 keV):

línea gamma [keV]	borde Compton E_max [keV]	área esperada [mV·ns]
1173.2	963.4	1216
1332.5	1118.1	1411



Izquierda: la lineshape teórica, con el escalón en E_max. Derecha: el mismo borde visto con tres resoluciones, y por qué el método clásico del 50% de altura está sesgado.

4. Del espacio de energía al espacio de canal

Archivo: `etapas/model.py`. **Función:** `forward_model(canales, E_gamma, A, edge_ch, sigma)`. Hace dos cosas:

```
1)  canal → energía:   E_e = canal · (E_max / edge_ch)
2)  convolución:       S(canal) = dσ/dE_e(E_e) ⊗ G(σ)
```

El mapa es lineal **por el origen**: área cero ↔ energía cero, sin offset. Es la Ec. 8 del paper y una decisión de diseño que todo el código respeta. Si algún día aparece un pedestal real, se agrega explícitamente como parámetro, no por la puerta de atrás dejando libre la ventana de ajuste.

- El kernel gaussiano está **normalizado a suma 1**, así que conserva área: subir σ ensancha y baja el pico sin cambiar las cuentas totales, y eso desacopla `A` de σ.
- El radio del kernel se capa a `(len(señal)−1)//2`, porque `np.convolve(mode="same")` devuelve el largo del **kernel** si el kernel es más largo que la señal — lo que pasa cuando σ se infla durante un ajuste que va mal.

> **La trampa del padding:** la convolución **no** puede evaluarse sólo dentro de la ventana de ajuste. `np.convolve` rellena con ceros, así que el modelo pierde los eventos de más abajo del límite izquierdo que el broadening debería meter hacia adentro, y el ajuste lo compensa **achicando σ**: en la prueba sintética de dos bordes daba σ un 18% bajo y D un 3.9% corrido. Por eso `forward_model` tiene `pad=True` por defecto: extiende la grilla hasta el canal 0 y 4σ a la derecha, convoluciona, y recorta. En la interfaz se puede destildar para ver el efecto en vivo.

Los tres métodos clásicos, y por qué no alcanzan

Archivo: `etapas/clasicos.py`. El paper no sólo propone NCC-SA: mide cuánto mejor es que dos métodos convencionales, y esa comparación está implementada:

- **50% de altura** (`model.punto_mitad_altura`): se lee directo sobre los datos el canal donde el espectro cae a la mitad del máximo.
- **Método de la diferencia** (`metodo_diferencia`): el borde es el canal donde la derivada es más negativa. La ventana de suavizado se deduce del ancho del propio flanco, porque con una constante el pozo de la derivada queda chato cuando el borde es ancho y el mínimo lo elige el ruido.

- **FWHM gaussiano** (`metodo_fwhm_gauss`): se ajusta el flanco con un escalón borroneado por una gaussiana, `C/2·erfc((ch−B)/(√2σ))` más un fondo lineal. Es lo que se obtiene de convolucionar un escalón con la resolución, así que su punto de media altura cae en B por construcción y el mismo ajuste entrega el FWHM.

Si el borde fuera un escalón perfecto los tres darían lo mismo. Sobre un borde sintético de posición **conocida**, los tres **sobrestiman**, y el sesgo crece con la resolución:

σ/B	50% de altura	diferencia	FWHM gaussiano	diferencia SIN ruido
0.03	+0.8%	+0.7%	+2.6%	+0.4%
0.06	+2.2%	+1.5%	+4.7%	+1.4%
0.09	+3.8%	+2.8%	+6.3%	+2.6%
0.12	+5.4%	+4.4%	+7.5%	+3.8%
0.18	+8.5%	+7.4%	+9.2%	+6.2%

La última columna es el mismo método sobre el espectro **sin ruido**: va igual de +0.4% a +6.2%, o sea **el sesgo es del método, no de la estadística**. La causa es común a los tres: la lineshape de Klein-Nishina no es un escalón, tiene una acumulación creciente antes del borde, y ninguno de los tres la contiene. Ésa es la justificación cuantitativa de ajustar la lineshape entera, que es lo que hace todo el resto de este documento.

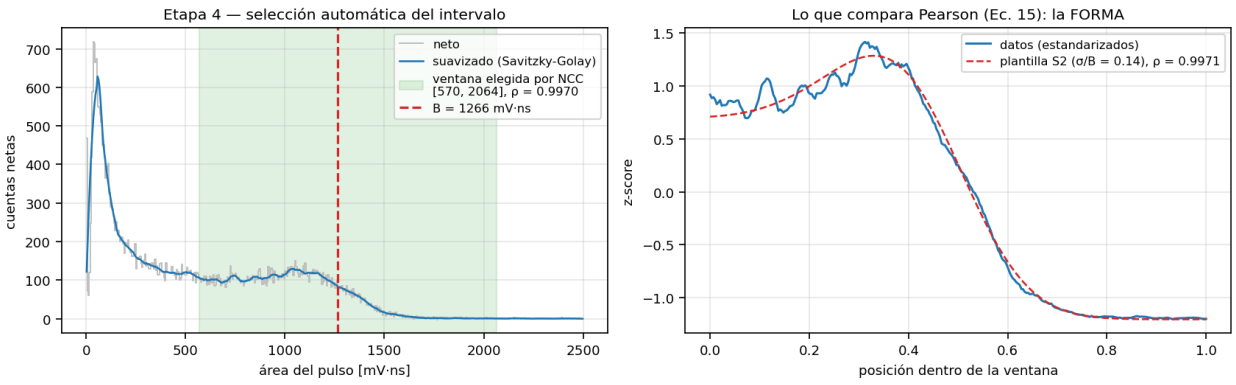
> **Una trampa al usarlos:** hay que darles una región **alrededor del borde**, no la ventana de ajuste del modelo extendido. Esa ventana arranca en ~0.25·B a propósito porque ese modelo contiene el exceso de bajo canal; los clásicos no lo contienen, y la caída del exceso es mucho más empinada que el borde. Medido sobre el Co-60 real: con la ventana de ajuste el mínimo de la derivada se va al exceso y da D = 0.363 en vez de 1.265, un −71%. Por eso `comparar` toma el borde del NCC como ancla.

5. Elegir la ventana de ajuste sin criterio humano (NCC)

Archivo: `etapas/ncc.py`. Es la Sección II-B del paper y, en la práctica, lo más útil que aporta: reemplaza «elegir a ojo dónde empieza y termina el borde» por un criterio reproducible.

La idea: se desliza una ventana sobre el espectro suavizado, se reinterpola a un número fijo de puntos y se calcula el **coeficiente de Pearson** (Ec. 15) contra la plantilla teórica. Pearson es inmune a offset y a escala, así que compara **forma**, no amplitud. Gana la ventana con ρ más alto.

Sobre estos datos: ventana **[570, 2064] mV·ns** con **ρ = 0.9970**, y de ahí sale la estimación del borde **B = 1266 mV·ns**.



Izquierda: la ventana elegida sobre el espectro real. Derecha: los dos perfiles estandarizados que Pearson compara.

Dos desviaciones del paper, ambas forzadas por los datos

1. El barrido 2D del paper está sobre-parametrizado respecto de su propio modelo. Si el mapa canal↔energía pasa por el origen (Ec. 8), fijado B la ventana ya está determinada: `[inicio_rel·B, fin_rel·B]`. Barrer inicio y fin libres agrega un grado de libertad que el modelo no tiene, y sobre datos reales se paga: el barrido libre eligió una ventana de 32 bins en la cola vacía con ρ = 0.984. Por eso el modo por defecto es `modo="origen"`, que barre **sólo B** — más robusto, 7× más rápido, y sobre sintético recupera B exacto. El barrido del paper sigue disponible con `modo="libre"`.

2. El número de puntos de la reinterpolación. El paper usa 100. La ventana real tiene ~570 bins y el suavizado Savitzky-Golay de 21 bins deja sólo ~27 puntos independientes, así que subir el número no agrega información — pero **bajarlo infla ρ**, porque la interpolación promedia ruido. Y ρ es justamente el criterio con que se comparan ventanas. El default acá es `n_puntos="auto"`: un punto por bin, sin inventar puntos. El número queda fijo durante todo el barrido, porque si cambiara con B los ρ dejarían de ser comparables.

> **El límite real del NCC:** compara forma, y no tiene manera de saber que a bajo canal hay física que la plantilla no contiene. Sobre estos datos eligió un inicio que se comía parte del exceso de bajo canal. **La selección automática resuelve la subjetividad, no la validez del modelo.** Es la razón por la que el paso 9 modela ese exceso en vez de confiar en que la ventana lo esquive.

De yapa, el NCC entrega la **semilla** del ajuste [fuera del paper]: la ventana ganadora queda mapeada punto a punto contra la plantilla, así que de ahí salen B, σ y D_0. Eso resuelve en la práctica la fragilidad del optimizador local.

6. El ajuste analítico

Archivos: `etapas/fit_single.py` (un borde) y `etapas/fit_double.py` (dos bordes solapados). Cuál se usa lo decide la fuente elegida.

6.1 Un solo borde

`ajustar_borde_unico` corre `least_squares` sobre {A, edge_ch, σ} dentro de la ventana. Con una sola línea gamma, D sale directo: `D = B / E_max`. Es el caso fácil, y por eso una fuente de una sola línea vale tanto para verificar.

> **Es frágil por naturaleza:** se mapeó el paisaje de la función de costo y el mínimo global sí está en el B verdadero, pero desde un arranque malo el optimizador queda ~10% off con σ inflado. Mitigación: sembrarlo con la semilla del NCC, que en la prueba con verdad conocida bajó el error en B de −26 canales a −0.2 sobre 600. La solución robusta es el paso 7.

6.2 Dos bordes solapados

Con dos líneas gamma (Co-60, Na-22) el espectro tiene dos bordes que se superponen. `modelo_dos_bordes` implementa las Ec. 11-13, con el segundo borde amarrado al primero:

```
CE_ch,2 = B1 + D·ΔE          θ = {A1, A2, B1, σ1, σ2, D}
```

> **Errata del paper (2 de 3):** la Ec. 12 define ΔE como la diferencia «entre las líneas gamma». Está mal: D convierte energía del **electrón** a canal, y lo que se separa en canal son los **bordes**. Para Co-60 son 154.7 keV y no 159.3. Con el ΔE de los gammas, D sale ~3% corrido. En el código, `fit_double.delta_E`.

Con la fuente cargada, los dos bordes están a 16% uno de otro (963.4 y 1118.1 keV).

El problema de fondo: en el Co-60 la separación entre los bordes es comparable a σ, así que la descomposición interna no tiene información aunque la posición del conjunto sí. Barriendo variantes, B1 se mueve ±0.5% pero D libre se mueve ±8%. Por eso se ajusta con

`ligar_origen=True` (D = B1/E_max,1), que da D estable en todas las variantes, y está justificado porque el test del origen sale limpio: el offset extrapolado a E = 0 es −2.3% de B1.

> **Para mejorar:** una fuente con los bordes bien separados (Na-22 los tiene en 341 y 1062 keV) haría honesta la descomposición de dos componentes, en vez de tener que ligarla.

7. El optimizador global (recocido simulado)

Archivo: `etapas/sa.py`. Es la Sección II-A del paper: Metropolis con enfriamiento geométrico, para escapar de mínimos locales.

Con 3 parámetros gana claro. Desde un arranque deliberadamente malo ($B_0 = 0.70 \cdot B$, $\sigma_0 = 3\sigma$), `least_squares` da B = 343 sobre un valor verdadero de 600, y el recocido da **600.2** con cualquiera de los dos objetivos.

> **Dos constantes del paper NO son transferibles:** Metropolis compara la temperatura con ΔE, así que T sólo tiene sentido **en las unidades del costo**. Medido acá, la calibración automática pide T_init ≈ $2 \cdot 10^9$ con la Ec. 14 (suma de cuadrados) y ≈ 3.2 con el error NCC: nueve órdenes de magnitud de diferencia contra el T_init = 2000 del paper. Lo mismo con su corte absoluto |ΔE| < 0.001. Por eso `recocido_simulado` usa `T_init="auto"` (calibra para ~80% de aceptación inicial) y tolerancia relativa.

> **Errata del paper (3 de 3):** la Ec. 14 dice que el recocido minimiza mínimos cuadrados, pero el texto de Metropolis dice que el costo es «NCC-based matching error». Hay que elegir uno. Se verificó que con el objetivo NCC las amplitudes son **indeterminables** (multiplicar A por 7.3 cambia el costo en exactamente 0), así que la Ec. 14 es la lectura utilizable.

Su aporte real fue diagnóstico. Al buscar de verdad el óptimo global mostró que Θ = {A1, A2, B1, σ1, σ2, D} está **sub-determinado**: cada libertad de más el óptimo la usa para algo sin sentido físico (offset de −5928 mV·ns en E = 0; σ1 > σ2 cuando la resolución tiene que crecer con E; A1 = 0 cuando el Co-60 emite las dos líneas casi igual). La conclusión no fue cambiar de optimizador sino **restringir el modelo con física**, que es lo que hace el paso siguiente.

El recocido es estocástico: una sola corrida no es un resultado. Se usa `sa_multiarranque`.

8. Forward folding con la simulación de Geant4

Archivo: `ajuste/sim_template.py`. Cambia la plantilla analítica por el espectro simulado, dejando el resto del esquema igual:

```
S_model(ch) = A · Σ_j  h_j · G(ch ; D·E_j , D·σ_E(E_j))
σ_E(E) = √(α²·E + β²)          [α: estadística de fotoelectrones]
```

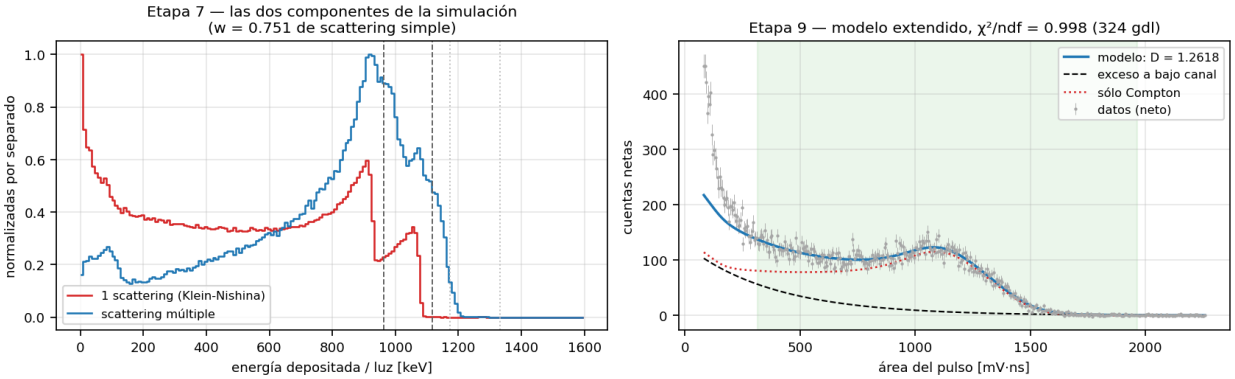
Qué contiene la simulación que la fórmula analítica no: scattering Compton **múltiple**, escape de los gammas dispersados (depende del tamaño del centellador), el quenching de Birks a baja energía y la eficiencia relativa de las dos líneas. Todo eso el ajuste analítico lo tiene que absorber inflando σ.

La simulación cargada acá genera 1173.2 keV (51%), 1332.5 keV (49%), y eso se **verifica** contra la fuente elegida antes de usarla.

> **Un modo de falla silencioso que ahora está tapado:** la plantilla **es** la fuente. Ajustar un espectro de Cs-137 contra una simulación de Co-60 no produce ningún error: produce un D perfectamente formateado y equivocado, porque la plantilla tiene sus bordes en otro lado. `sim_template.lineas_primarias` lee la rama `E0_keV` y la interfaz bloquea el forward folding si no coincide con la fuente elegida.

Dos escalas que **no hay que mezclar**: la rama `Lvis_keVee` es la luz visible con Birks ya aplicado —lo que ve el SiPM— y da keVee; la rama `Edep_keV` es la energía depositada y da keV. Su cociente reproduce el quenching de la simulación, pero eso **no** es un control independiente: Lvis es Edep multiplicada por el quenching, así que ajustar los mismos datos con las dos plantillas tiene que dar ese cociente.

Detalle de implementación que importa para la velocidad: el binning de la plantilla es **irrelevante** para el resultado, porque el kernel gaussiano es ~90 veces más ancho que un bin. Rebinnear de 700 a 175 bins deja D igual en la cuarta cifra y acelera 6×, que es lo que hace viables el recocido y el MCMC.



Izquierda: las dos componentes de la simulación, separadas por `NCompton`*. El scattering múltiple deposita energía* **por encima** *del borde, y ése es exactamente el relleno que la plantilla analítica no tiene. Derecha: el ajuste del paso 9 sobre los datos.*

9. El modelo extendido: los sistemáticos como parámetros

Archivo: `ajuste/modelo_ext.py`. Es el resultado vigente y la idea central del análisis actual.

Hasta acá quedaban dos incertidumbres que se trataban **a mano**: cuánto scattering múltiple hay (se reportaba la diferencia entre plantillas como sistemático) y el exceso de bajo canal (se resolvía cortando la ventana, lo que tira estadística y deja una dependencia con dónde se corta). Las dos entran ahora al modelo:

```
S(ch) = A·[ w·h_1scatt + (1−w)·h_múltiple ] ⊗ G(σ_E)  +  C·exp(−ch/λ)
σ_E(E) = √( a²·E² + α²·E + β² )
```

`w` es cuánto scattering simple **piden los datos**, y el término exponencial es el exceso. Θ = {A, D, α, w, C, λ, β, a}; se ajusta el subconjunto que uno declare libre.

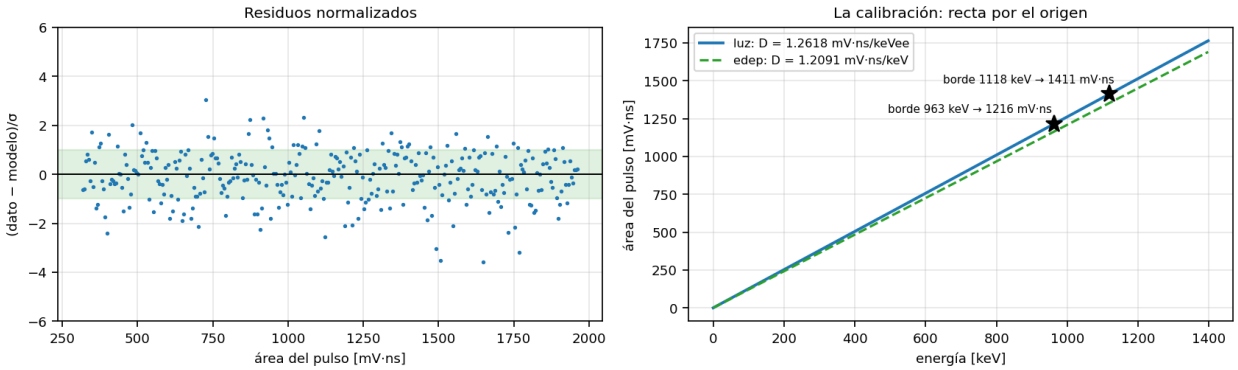
Sobre estos datos, con la ventana [317, 1963]: **w = 0.797 ± 0.072** contra **0.751** de la simulación, o sea coinciden dentro de 0.6σ. El χ²/ndf es 0.998 con 324 grados de libertad.

Que la geometría simulada reproduzca cuánto scattering múltiple hay es un resultado en sí mismo. Y `w` está realmente medido, no es decorativo: fijarlo en el valor de la simulación no cambia el χ², mientras que fijarlo en 1 —que es lo que asume la plantilla analítica del paper— lo empeora en Δχ² ≈ +13 y sube D un 3.4%.

Lo que se gana: el error de D deja de ser «la dispersión entre tres ajustes que no se pueden comparar» y pasa a ser el ancho marginal del posterior, que ya incluye la ignorancia sobre `w` y sobre el exceso vía la correlación corr(D, w) ≈ +0.96. Eso sí es propagable.

Lo que **no** se gana: si la simulación no contiene un proceso, `w` no lo inventa. `w` mide cuánto pesa el scattering múltiple **simulado**, no valida la geometría.

El χ² de Pearson, no el de Neyman: calcular la barra de error con las cuentas **observadas** sesga D hacia abajo, porque un bin que fluctuó bajo recibe σ más chico y por lo tanto más peso. Con la estadística de este experimento (~15-25 cuentas/bin) el sesgo era comparable al error estadístico que se reportaba. Medido sobre 20 realizaciones con verdad conocida: Neyman da −0.53% de sesgo y cobertura 8/20; Pearson da −0.06% y 15/20, que es lo que corresponde. Por eso `spectrum.varianza_predicha` usa la varianza del **modelo** y el ajuste itera tres veces, ya que esa varianza depende de los parámetros.



Izquierda: residuos normalizados del ajuste — planos, o sea el exceso no está tapando estructura del borde. Derecha: la recta de calibración resultante, con los bordes de esta fuente.

10. El posterior (MCMC)

Archivo: `ajuste/mcmc.py`. Metropolis-Hastings propio, sin dependencias externas.

- La propuesta usa la **covarianza del ajuste** escalada por 2.4²/d, que es el óptimo para un objetivo gaussiano en d dimensiones. Así la cadena viaja **a lo largo** de la degeneración D–α en vez de chocarse contra ella.
- Las cadenas arrancan **dispersas**. No es un detalle: R-hat compara la varianza entre cadenas contra la interna, y si todas arrancan en el mismo punto R-hat da ~1 aunque ninguna haya explorado nada.
- Diagnósticos: `gelman_rubin` (R-hat), `autocorr_integrada` (τ y por lo tanto n_eff) y la fracción de aceptación.
- Cuando la varianza depende de los parámetros (Pearson), el posterior lleva el término `ln σ²(θ)`, que con varianza fija era constante y se podía ignorar.

`python codigo/ajuste/mcmc.py` corre el test con **verdad conocida**: genera datos con la simulación a D y α conocidos, con ruido de Poisson a la estadística real de este experimento, y verifica que los parámetros caen dentro del 68% del posterior. Ahí se ve también el sesgo de Neyman en vivo.

Además se hizo un **cierre con verdad conocida** de la cadena completa: 20 realizaciones Poisson a la estadística real dan sesgo en D de −0.10%, rms 0.0116 contra un error reportado de 0.0127, y cobertura 17/20.

11. El resultado y su presupuesto de error

Archivos: `calibracion.py` (el número congelado del Co-60, que es lo que importan otros proyectos) y la pestaña ★ **Calibración** de `gui.py`, que lo recalcula sobre los archivos y la fuente que estén cargados.

Sobre estos datos, la conversión es:

```
E [keVee] = área [mV·ns] / 1.2618          área [mV·ns] = E [keVee] × 1.2618
σ_E(E) = 2.764 · √E  keV        (E en keV)
```

De dónde sale cada término del error

La pestaña de Calibración **mide** el presupuesto en vez de citarlo: corre las variantes sobre los mismos datos y compara.

término	cómo se mide	cuánto da (Co-60, este detector)
estadística	ancho del posterior; ya incluye w y el exceso por correlación	≈ 1.1%
ventana	barrido del límite inferior de la ventana de ajuste	≈ 0.9%
forma del exceso	exponencial contra ley de potencia: describen los datos igual de bien	**≈ 2.2-2.4% ← el que manda**
definición del área	barrido de compuerta (45/55/70/90 ns), onset (5/10/20%) y descarte	≈ 1.0%
total	suma en cuadratura	≈ 2.8-2.9%

El único término que se **arrastra** en vez de medirse en vivo es el de la definición del área, porque cada variante obliga a re-parsear los XML (~40 s cada una).

> **Lo que esto dice: el sistemático domina y ya se sabe cuál es.** La forma que se le asume al exceso de bajo canal no se cierra ajustando mejor: exponencial y ley de potencia dan χ^2/ndf indistinguibles y D con ~2.3% de diferencia. Hay que **medir** el exceso.

Cada resultado se guarda en `.cache/calibraciones.json` con una clave que incluye la fuente y la huella de los archivos, y la pestaña superpone las fuentes ya medidas. Ojo con qué prueba eso: cada fuente entregó su D con el mapa por el origen, así que su punto cae exacto sobre su propia recta. **Lo que hay que mirar es la dispersión entre fuentes**, y eso es el test de linealidad.

12. La interfaz

Archivo: `gui.py`. Catorce pestañas, una por etapa, cada una con controles a la izquierda, gráfico al centro y una explicación a la derecha. No muestra PNGs guardados: **recalcula**, que es lo que permite mover un parámetro y ver qué se rompe.

pestaña	qué permite ver
Archivos	elegir muestra, fondo, simulación y **fuente**; diagnóstico de tiempo vivo y canal; verifica que la simulación corresponda a la fuente
0 · Espectro	el neto, con selector `gate/legacy` para ver el antes/después del arreglo del área
1 · Klein-Nishina	la lineshape teórica y los bordes
2 · Forward model	el efecto de σ y el sesgo del método del 50%; destildar «pad» muestra la trampa de la convolución
3 · Un borde	el ajuste local y su fragilidad
4 · NCC	la selección automática de ventana; modo «libre» muestra por qué el barrido 2D del paper falla
6 · Dos bordes / Un borde	el modelo que corresponda a la fuente; destildar las ligaduras muestra la sub-determinación
5 · Recocido	el óptimo global contra el local
7 · Geant4	las plantillas simuladas
8 · NCC→SA→MCMC	la cadena entera y el posterior; la distancia entre plantillas **es** el sistemático
9 · El área	el hallazgo del paso 1, en cuatro paneles
10 · Modelo ext.	el modelo del paso 9, con w y el exceso libres
★ Calibración	el resultado sobre los datos cargados, con el presupuesto de error medido y el registro por fuente
? Parámetros	glosario: qué es B, σ, D, ρ, Θ, α, keVee vs keV…

Los cálculos pesados corren en un hilo aparte y hay una barra de progreso abajo: indeterminada cuando el paso no sabe cuánto falta, y con porcentaje y tiempo restante cuando sí (el parseo de los XML mide bytes leídos; la pestaña de Calibración cuenta los ajustes antes de empezar; los MCMC reportan cadena y paso).

13. Qué revisar y qué mejorar

En orden de impacto sobre el número final.

1. Medir el exceso de bajo canal — es el sistemático dominante

Hoy vale ~2.3% de D y no se cierra con más estadística ni con mejor ajuste: exponencial y ley de potencia describen los datos igual de bien. Lo que lo resuelve es **una corrida corta con la fuente apantallada, o movida de lugar**, que diga si viene de dispersión en el material de alrededor. Es la medición de mayor rendimiento pendiente.

2. Una segunda fuente, para dejar de asumir la linealidad

Con una sola fuente la recta se traza con un punto y el origen. El registro por fuente ya existe y la interfaz superpone lo que haya; falta la medición. Para cerrarla bien hace falta también una **simulación de Geant4 de esa fuente** con esta geometría: sin ella queda la ruta analítica, con su sesgo conocido por no tener scattering múltiple.

3. Recuperar el tiempo vivo

Con ~40% de tiempo vivo se tiró el 60% de la estadística. Bajar la tasa de trigger o acelerar el guardado da un factor √2.5 en el error estadístico sin tocar nada del análisis.

4. La definición del área

Vale ~1.0% y hoy se arrastra del barrido de la Etapa 9 en vez de re-medirse. Sería razonable hacer que el barrido se pueda correr desde la interfaz (es lento porque re-parsea, pero se puede cachear una vez por definición). También queda abierto qué hacer con el pedestal de los eventos sin pre-pulso (bandera 8): hoy se marca y se acepta.

5. Cosas menores pero reales

- El término `a·E` y el β de la resolución se van a cero: con dos bordes tan cercanos no hay palanca para separar los tres términos. Con una fuente de energía muy distinta sí la habría.
- El ajuste de dos bordes tiene que ligar el origen para ser estable. Con una fuente de bordes separados dejaría de hacer falta.
- La simulación se cargó con la geometría actual; si cambia el montaje (portafuente, blindaje, distancia), `w` lo va a notar antes que cualquier otra cosa. Vale la pena mirarlo después de cada cambio mecánico.

Fin. Este documento se regenera con `python codigo/documentacion.py`*; los números y las figuras se recalculan cada vez, así que refleja el código del día en que se corrió.*